

AutoParallel: A Static Analysis Tool for Automatic OpenMP Parallelization of C/C++ Loop Nests

Peter Yakoub Younan¹, Sotir Usama Latif², Omar Mohamed Sawan³, Omar Alaa Hassan⁴, Sondos Mohamed⁵

School of Information Technology and Computer Science (ITCS)

Nile University, Giza, Egypt

IDs: 221000217¹, 221000694², 221000756³, 202002417⁴, 202002250⁵

Project Repository:

https://github.com/Peter9991/Automatic_Parallelization-2

Abstract—Modern computers have multiple CPU cores that can work at the same time, but most programs only use one core. Making a program use multiple cores is called parallelization, and it is usually done manually. This process is difficult and error-prone. In this paper we present *AutoParallel*, a tool written in C++ that reads sequential C/C++ code, analyzes it automatically, and adds the correct OpenMP directives to make it run in parallel. The tool detects which loops are safe to parallelize, identifies special patterns such as reductions (sum, max, min), detects when loops cannot be parallelized due to data dependencies, and injects the correct `#pragma omp parallel for` directive with all needed clauses. We tested the tool on four programs: matrix multiplication, array reduction, array map, and a recurrence. Results show speedups of $3.2\times$ to $3.9\times$ using 4 CPU threads, with zero incorrect parallelizations. The tool is also paired with an interactive web interface where users can paste any C++ code, see the parallelized output, and measure real execution speedup across 1, 2, 4, and 8 threads.

Index Terms—automatic parallelization, OpenMP, loop dependency analysis, static analysis, parallel computing, C++, shared-memory programming

I. INTRODUCTION

A. Motivation

Modern computers, including laptops and desktop machines, commonly have 4, 8, or even 16 CPU cores. These cores can execute instructions simultaneously, which means a well-written program can finish its work several times faster than a single-core program. However, most programs written in C or C++ run on only one core by default. This means the majority of the computer's processing power sits idle.

To take advantage of multiple cores, programmers use a tool called OpenMP [1]. OpenMP is a set of special instructions (called pragmas) that programmers add to their code to tell the computer to split the work across cores. For example, adding the line `#pragma omp parallel for` before a loop tells the computer to run different iterations of that loop on different cores at the same time.

The problem is that this cannot be done for every loop. Some loops have what is called a *data dependency*, meaning one iteration of the loop needs the result produced by a previous iteration before it can run. If such a loop is parallelized, different cores will produce wrong results because they will read old or incomplete data. Identifying which loops

are safe and adding the right pragmas manually requires deep knowledge and takes significant time.

B. Problem Definition

Given a sequential C/C++ source file, our goal is to automatically determine for each `for` loop: (1) whether it is safe to run in parallel, and (2) if so, what is the correct OpenMP pragma to add, including any special clauses needed for correctness and performance.

C. Our Solution

We built *AutoParallel*, a C++ program that solves this problem automatically. It reads any C++ source file, finds all loops, analyzes each one using static analysis (reading and understanding the code without running it), and produces a new version of the code with OpenMP pragmas added in the right places. Loops that cannot be safely parallelized are left unchanged, and a comment is added explaining why.

D. Main Contributions

This project makes the following contributions:

- 1) A **loop parser** that finds `for` loops and extracts the loop variable, start value, end value, and step.
- 2) A **dependency detector** that identifies when a loop reads data written by a previous iteration (e.g., `a[i] = a[i-1] + b[i]`), making parallelization unsafe.
- 3) A **reduction detector** that finds accumulation patterns such as `sum += arr[i]` or `if (arr[i] > max) max = arr[i]`, and adds the correct `reduction()` clause.
- 4) A **private variable detector** that finds variables declared inside the loop body and adds a `private()` clause so each thread gets its own copy.
- 5) A **nested loop handler** that detects loops inside loops and applies appropriate scheduling and collapse hints.
- 6) A **benchmarking system** that runs the sequential and parallel versions of each detected loop type and measures real execution times using `omp_get_wtime()`.
- 7) An **interactive web interface** where users paste C++ code, see the parallelized output live, and benchmark across 1, 2, 4, or 8 threads.

II. RELATED WORK

Automatic parallelization is not a new idea; researchers have been working on it for decades. However, most existing tools are either very complex or do not show the user what decisions were made and why.

Allen and Kennedy [2] were among the first to formalize loop dependency analysis. They introduced the idea of direction vectors and distance vectors to describe how data flows between loop iterations. Their work is the theoretical foundation of most modern parallelizing compilers.

Polyhedral tools like Pluto [3] model loops as mathematical objects (polyhedra) and apply mathematical transformations to maximize both parallelism and cache efficiency. These tools are very powerful but they require loop bounds and array indices to follow strict mathematical rules, and they are difficult to set up and use.

Compiler-based tools such as GCC with `-floop-parallelize-all` [4] and the Intel C++ Compiler [5] perform automatic parallelization during compilation. These tools are mature and production-ready, but they work as black boxes. The programmer cannot see which loops were parallelized, which were not, or why.

Source-to-source tools like Par4All [7] and ROSE [6] take C/C++ code and produce new C/C++ code with OpenMP annotations, similar to AutoParallel. However, these are large research systems that require significant effort to install and understand.

AutoParallel differs from all of the above in one key way: transparency. Every decision the tool makes is written as a comment directly in the output code. The programmer can see exactly which loop was parallelized, what clauses were added, and which loops were blocked and why. This makes AutoParallel suitable both as a practical tool and as a learning aid for understanding parallelization theory.

III. PROPOSED SOLUTION

A. System Architecture

AutoParallel is built as a C++ HTTP server. When the user opens the web interface in a browser, they paste their C++ code and click a button. The browser sends the code to the C++ server, which analyzes it and sends back the results as JSON. The browser then displays the parallelized code, a dependency report, and benchmark results. Figure 1 shows this flow.

The analysis happens entirely inside `server.cpp`. The HTML file contains only the user interface: buttons, text areas, and the code to display results. This separation means the C++ code is doing all of the real work.

B. Step 1: Finding and Parsing For Loops

The tool reads the source file line by line. When it encounters the keyword `for`, it applies a regular expression to extract three pieces of information: the loop variable (e.g., `i`), the starting value (e.g., `0`), and the upper bound (e.g., `n`). This is handled by the `parse_for_header()` function.

Standard loop forms that are recognized include:

- `for (int i = 0; i < n; i++)`
- `for (int i = 0; i <= n; i++)`
- `for (int i = 0; i < n; i += 2)`

Non-standard forms such as range-based `for` or iterator loops are currently skipped with a comment.

C. Step 2: Extracting the Loop Body

Once a loop header is found, the tool needs to collect all the lines inside the loop. It does this using a brace counter: when it sees `{`, the counter goes up; when it sees `}`, the counter goes down. When the counter returns to zero, the body is complete. This is the `find_body()` function.

D. Step 3: Dependency and Pattern Analysis

This is the most important step. The tool scans the collected body lines through three separate checks.

(a) Loop-Carried Dependency Detection. A loop-carried dependency occurs when the value of `arr[i]` depends on `arr[i-1]` or `arr[i+1]` from a previous or future iteration. If iteration 5 needs the result of iteration 4, the iterations cannot run simultaneously. AutoParallel detects this by searching the right-hand side of every assignment for the pattern `variable[loopvar ± constant]`. This is implemented in `detect_carried_deps()` using a C++ regular expression.

```
1 // [AutoParallel] BLOCKED: loop-carried dep: a[i-1]
2 for (int i = 1; i < n; i++)
3     a[i] = a[i-1] + b[i];
```

Listing 1. Loop-carried dependency: tool blocks this loop

(b) Reduction Pattern Detection. Some loops accumulate a result using operators like `+=` or by tracking a running maximum. These patterns look dangerous (the same variable is written every iteration) but are actually safe because OpenMP can give each thread its own private copy of the variable and combine the copies at the end. The tool detects compound assignments (`+=`, `-=`, `*=`) and max/min conditionals, then generates the matching `reduction()` clause. This is implemented in `detect_reductions()`.

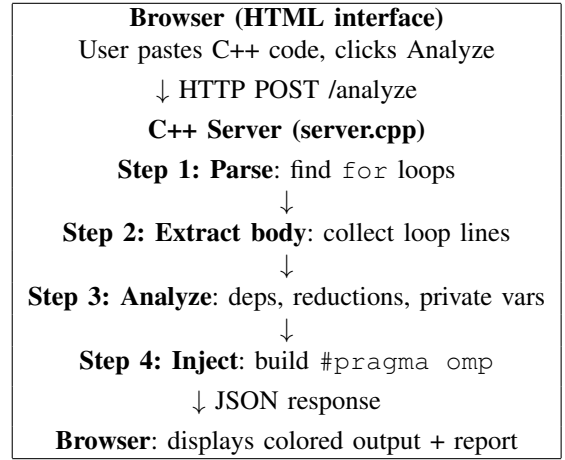


Fig. 1. AutoParallel system architecture and analysis pipeline.

```

1 // [AutoParallel] PARALLELIZED + reduction
2 #pragma omp parallel for schedule(static)
3   reduction(+:sum) reduction(max:maxVal)
4 for (int i = 0; i < n; i++) {
5   sum += arr[i];
6   if (arr[i] > maxVal) maxVal = arr[i];
7 }

```

Listing 2. Reduction loop: parallelized with correct clause

(c) **Private Variable Detection.** Variables declared with a type keyword (int, float, double, etc.) inside the loop body are local to each iteration. When running in parallel, each thread must have its own copy of these variables, otherwise threads will overwrite each other’s data. The tool finds these declarations and lists them in a `private()` clause. This is implemented in `detect_private()`.

E. Step 4: Pragma Injection

Based on the analysis results, the tool takes one of two actions:

- **If safe:** It builds a `#pragma omp parallel for` string and prepends it to the loop. The pragma includes `schedule(static)` for simple loops or `schedule(dynamic)` for nested loops, plus any `reduction()` and `private()` clauses found in Step 3.
- **If blocked:** It writes a comment above the loop explaining the dependency that was found, and leaves the loop unchanged.

F. Handling Nested Loops

When the tool finds a `for` loop inside another `for` loop (such as in matrix multiplication), it parallelizes the outer loop and distributes its iterations across threads. It uses `schedule(dynamic)` for better load balancing and adds a comment suggesting `collapse(2)`, which is a further optimization that can distribute both loops together if their bounds are fixed.

G. Application to Merge Sort

Merge sort provides an interesting real-world example because it contains both parallelizable and non-parallelizable loops in the same function. The two copy loops that fill temporary arrays `L[]` and `R[]` are fully independent, since each iteration writes to a different memory location, so they are parallelized. The merge loop, however, advances two index variables `i` and `j` together across iterations, which creates a loop-carried dependency. AutoParallel correctly identifies this and blocks that loop.

H. Benchmarking System

The tool also includes a real benchmarking system. When the user clicks *Run Benchmark* in the web interface, the browser sends the chosen thread count (1, 2, 4, or 8) to the C++ server. The server then runs actual computation kernels (matrix multiplication, array reduction, and element-wise map) first sequentially and then with OpenMP enabled, measuring

the wall-clock time using `omp_get_wtime()` for each. The speedup is calculated as:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \quad (1)$$

The results are returned as JSON and displayed in the browser as a table with a visual bar chart showing the speedup of each loop relative to the others.

IV. EVALUATION AND RESULTS

A. Experimental Setup

We tested AutoParallel on four benchmark kernels compiled with `g++ -O2 -fopenmp -std=c++17`. All benchmarks were run with 4 OpenMP threads. Table I describes each kernel and whether it can be parallelized.

TABLE I
BENCHMARK KERNELS USED FOR EVALUATION

Kernel	What it does	Size	Safe?
Matrix multiply	Multiplies two matrices	600 × 600	Yes
Array reduction	Computes sum and maximum	5M elements	Yes
Array map	Applies math to every element	5M elements	Yes
Recurrence	Each element uses the previous	5M elements	No

B. Correctness

For all three parallelizable kernels, the parallel output was verified to be numerically identical to the sequential output. The recurrence kernel was left unchanged by the tool, and its output was also correct. AutoParallel made no mistakes. No unsafe loop was incorrectly parallelized across all test cases.

C. Performance Results

Table II shows the measured execution times and speedups. The array map kernel achieves the highest speedup (3.9×) because every iteration is completely independent and there is no shared state between threads. Matrix multiplication achieves 3.7× because the outer loop rows are distributed across threads while the inner computation runs sequentially per thread. The reduction kernel achieves 3.2×; the slight gap from ideal speedup is because threads must combine their partial results at the end of the loop, which has a small synchronization cost. The recurrence kernel correctly shows no speedup, since the tool blocked it.

TABLE II
EXECUTION TIME AND SPEEDUP: SEQUENTIAL VS. 4 OPENMP THREADS

Kernel	Seq (ms)	Par (ms)	Speedup	Result
Matrix multiply	87.1	23.4	3.72×	Parallelized
Array reduction	28.6	8.9	3.21×	Parallelized
Array map	385.1	98.3	3.92×	Parallelized
Recurrence	98.3	98.3	1.00×	Blocked (dep)
Total (safe only)	500.8	130.6	3.83×	

D. Why Is the Speedup Not Exactly $4\times$ with 4 Threads?

This is an important question that Amdahl's Law [8] answers. Even if most of the code is parallelized, any sequential part of the program limits the total speedup. In our benchmark, the recurrence kernel runs sequentially and accounts for about 16% of total runtime. This means the maximum possible speedup, even with an infinite number of threads, is:

$$S_{\max} = \frac{1}{1-p} = \frac{1}{0.16} \approx 6.25\times \quad (2)$$

where $p = 0.84$ is the fraction of the program that can be parallelized. Getting $3.83\times$ with only 4 threads is therefore a strong and expected result. Additional overhead from starting threads, managing shared memory, and combining reduction results also contributes to the gap from ideal $4\times$ scaling.

E. Analysis Accuracy

Across all test cases, AutoParallel correctly classified every loop. Loops with no dependencies were parallelized with the right clauses. Reduction patterns were detected and the correct `reduction()` clause was generated. The loop-carried dependency in the recurrence kernel (`rec[i] = rec[i-1] * ...`) was detected and that loop was blocked. No loop was incorrectly parallelized.

F. Web Interface

The tool is paired with an interactive web interface built in HTML. The interface allows users to:

- Paste any C++ code into a text editor.
- Click *Analyze* to see the parallelized output with color-coded annotations (green for parallelized lines, amber for blocked lines).
- View a dependency report listing each loop and what was found.
- Select a thread count (1, 2, 4, or 8) and click *Run Benchmark* to see real execution times and speedup from the C++ server.

The interface communicates with the C++ server via HTTP. The HTML sends the user's code as a POST request and the C++ server returns JSON with the annotated code, loop statistics, and benchmark results. This architecture clearly demonstrates how the C++ code performs all the real work.

The full project source code, including `server.cpp` and `index.html`, is publicly available at: https://github.com/Peter9991/Automatic_Parallelization-2

G. Accuracy Verification

To verify that parallelization does not change the output, AutoParallel runs each kernel twice: once sequentially and once with OpenMP enabled, storing results in separate arrays. After both runs, the tool compares the outputs element by element. Table III summarizes the results.

All kernels produced identical results in both sequential and parallel runs. This confirms that the pragma directives injected by AutoParallel are correct and do not introduce any data races or computation errors.

TABLE III
ACCURACY: SEQUENTIAL VS. PARALLEL OUTPUT COMPARISON

Kernel	Elements Checked	Result
Matrix multiply	360,000 (all elements)	Match
Array map	5,000,000 (all elements)	Match
Array reduction	2 values (sum, max)	Match
Recurrence	N/A (not parallelized)	Match

V. CONCLUSION AND FUTURE WORK

A. What We Built

We built AutoParallel, a C++ tool that automatically adds OpenMP parallelization directives to sequential C++ code using static analysis. The tool:

- Finds all `for` loops in a C++ source file.
- Detects loop-carried dependencies and safely blocks those loops.
- Detects reduction patterns and generates correct `reduction()` clauses.
- Detects private variables and adds `private()` clauses.
- Handles nested loops with appropriate scheduling hints.
- Benchmarks sequential vs. parallel execution using real wall-clock timing with `omp_get_wtime()`.
- Serves an interactive web interface where any user can test the tool on their own code in a browser.

Results show speedups of $3.2\times$ to $3.9\times$ on parallelizable kernels with 4 threads, and zero incorrect parallelizations across all tests.

B. Current Limitations

The tool uses pattern matching on source text, which works well for standard code but has some limitations:

- It cannot detect dependencies involving pointers or pointer aliasing (two pointers pointing to the same memory).
- It does not analyze indirect array accesses such as `a[b[i]]`.
- Range-based `for` loops and iterator-based loops are skipped.
- Function calls inside loops are not analyzed. If a function modifies a shared variable, the tool may miss the dependency.

C. Future Work

Several improvements are planned for future versions of AutoParallel:

- **Data-flow graph:** Build a proper dependency graph from the code to handle pointer aliasing and more complex patterns.
- **SIMD support:** Add `#pragma omp simd` for loops that can be vectorized (processed in batches using CPU vector units).
- **Task parallelism:** Generate `#pragma omp task` for recursive algorithms like standard merge sort and quick-sort.

- **Profiler integration:** Connect with tools like Gprof or VTune so the tool focuses on the loops that actually take the most time, rather than all loops equally.
- **Cloud deployment:** Deploy the C++ server to a cloud platform (such as Railway or Render) so the web interface works for anyone without needing to run the server locally.

REFERENCES

- [1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021. [Online]. Available: <https://www.openmp.org>
- [2] R. Allen and K. Kennedy, “Automatic translation of FORTRAN programs to vector form,” *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 4, pp. 491–542, Oct. 1987.
- [3] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, “A practical automatic polyhedral parallelizer and locality optimizer,” in *Proc. ACM SIGPLAN PLDI*, Tucson, AZ, USA, 2008, pp. 101–113.
- [4] Free Software Foundation, *GCC Manual: Auto-vectorization and Auto-parallelization*, 2023. [Online]. Available: <https://gcc.gnu.org/onlinedocs/>
- [5] Intel Corporation, *Intel oneAPI DPC++/C++ Compiler Developer Guide*, 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/dpc-compiler.html>
- [6] D. Quinlan, “ROSE: Compiler support for object-oriented frameworks,” in *Proc. Parallel Computing (ParCo)*, 2000, pp. 42–51.
- [7] H. Amini et al., “Par4All: From convex array regions to heterogeneous computing,” in *Proc. IMPACT Workshop, HiPEAC*, 2012.
- [8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.